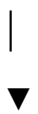
Ragnarok Online OCR Engineering Guide

Section 1 - System Architecture Redesign

Current Architecture:

Screen Capture



RapidOCR

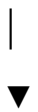
Overlay

This architecture is fundamentally limited.

No amount of PP-OCR tuning will produce Genshin-level results because the architecture itself lacks the layers used by modern computer vision systems.

The architecture should become:

DXGI Capture



Frame Manager

ROI Manager

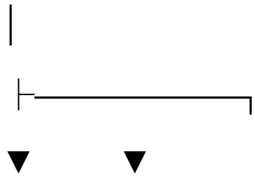
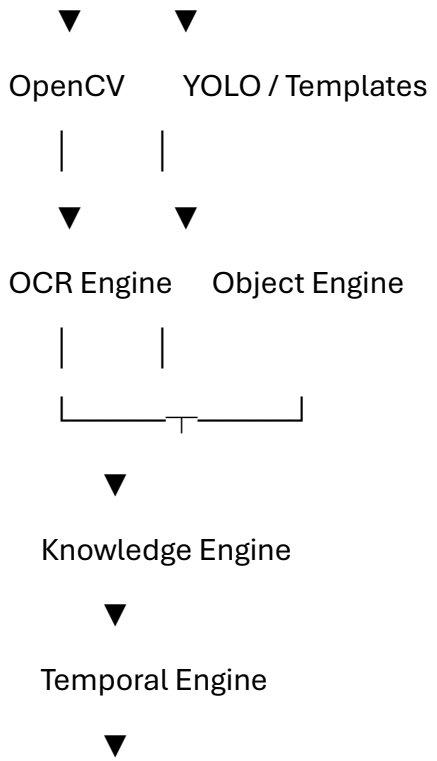


Image Pipeline Vision Pipeline



Why Current OCR Fails

Current OCR attempts:

Take Screenshot

↓

Find Text

↓

Recognize Text

RO contains many things that should never be OCR'd.

Examples:

HP Bar

SP Bar

Buff Icons

Skill Icons

Status Effects

Cast Bar

Target Marker

Party Icons

These should use:

Pixel Reading

Template Matching

YOLO Detection

instead.

Every object removed from OCR increases OCR accuracy.

Layer 1

Capture System

Current Goal:

Pixel Perfect Frames

Required:

DXGI Desktop Duplication

Avoid:

Graphics.CopyFromScreen()

BitBlt

GDI

because they introduce:

Blur

DPI Scaling

Color Conversion

Desktop Composition Artifacts

Expected Result:

1:1 GPU framebuffer

Layer 2

Frame Manager

Never OCR every frame.

Example:

Game:

120 FPS

OCR:

10 FPS

Overlay:

120 FPS

Most values do not change every frame.

Examples:

Character Name

Base Level

Job Level

Class

may stay unchanged for hours.

Cache these.

Suggested Class

```
public sealed class FrameCache
{
   public Mat CurrentFrame;

   public Mat PreviousFrame;

   public long FrameId;

   public DateTime Timestamp;
}
```

Layer 3

ROI Manager

Most OCR systems waste 95% of available compute.

Current:

1920x1080

↓

OCR

Bad.

Instead:

Character Panel

HP/SP Panel

Map Name

Target Name

Inventory

Chat

Monster Name

Party Window

Each becomes a dedicated OCR region.

Example:

```
public class RoiRegion
{
   public string Name;

   public Rectangle Bounds;

   public OcrProfile Profile;
}
```

Dynamic ROI Detection

Static coordinates eventually fail.

Different users:

Different Resolution

Different UI Scale

Different Skin

Different Window Size

Use anchor detection.

Example:

Find:

Basic Info Window

through:

Template Matching

Then derive:

HP Region

SP Region

Level Region

relative to anchor.

This makes OCR resolution independent.

Layer 4

OCR Profiles

Every region should have its own preprocessing pipeline.

Wrong:

One OCR Configuration

Correct:

Map Profile

Chat Profile

Inventory Profile

Monster Profile

Stats Profile

Example:

Map Name:

{
  "Upscale": true,
  "CLAHE": true,
  "Threshold": false,
  "Denoise": false
}

Inventory:

{
  "Upscale": true,
  "CLAHE": true,
  "Threshold": true,
  "Denoise": true
}

Monster Name:

{

"Upscale": true,

"CLAHE": false,

"Threshold": false,

"Denoise": false

}

Layer 5

OCR Routing Engine

One OCR engine is not enough.

Modern systems route OCR jobs.

Example:

HP/SP

↓

Windows OCR

Map Names

↓

PP-OCRv5

Chat

↓

Tesseract

Inventory

↓

PP-OCRv5

Monster Names

↓

PP-OCRv5 + Dictionary

Suggested Interface

```
public interface IOcrEngine
{
    OcrResult Read(Mat image);
}
```

Implementations:

RapidOcrEngine

WindowsOcrEngine

TesseractEngine

Layer 6

Knowledge Engine

This is where most game OCR projects become powerful.

OCR should not blindly trust text.

Example:

OCR reads:

Poyon Cave F1

Knowledge Engine knows:

Payon Cave F1

exists.

Apply correction.

Sources:

rAthena

mob_db.yml

item_db.yml

skill_db.yml

map_index.txt

job_db.yml

Build local dictionaries.

Layer 7

Temporal Engine

Single frame OCR is unreliable.

Store:

Previous 10 Frames

Example:

Payon

Payon

Payon

Poyon

Payon

Payon

Final:

Payon

Weighted Voting Formula

Final Score

=

Confidence

+

Occurrence Count

+

Dictionary Score

This alone often removes 80% of false positives.

Layer 8

Overlay Engine

Overlay should never display raw OCR.

Pipeline:

OCR

↓

Validation

↓

Dictionary Correction

↓

Temporal Correction

↓

Overlay

Current screenshot shows:

ClassName = ?

Monster = ?

HP = ?

These should never reach overlay.

Unknown values should remain cached until a valid replacement is found.

Example:

Previous:

Archer

Current:

?

Overlay:

Archer

Keep last valid value.

Never display OCR failures.

Ragnarok Online OCR Engineering Guide

Section 2 - Image Processing, OpenCV Pipeline, Super Resolution, and OCR Enhancement

---

Overview

Most OCR projects spend months tuning OCR models.

Most of the accuracy gains come before OCR ever runs.

A typical OCR pipeline:

Screenshot

↓

OCR

↓

Result

Professional computer vision pipeline:

Screenshot

↓

Preprocessing

↓

Region Enhancement

↓

Super Resolution

↓

Contrast Optimization

↓

Noise Reduction

↓

Text Isolation

↓

OCR

↓

Post Processing

For Ragnarok Online specifically:

80% of improvements

=

Image Pipeline

20% of improvements

=

OCR Model

Understanding RO Font Rendering

RO text is unusual.

A typical font:

Black Text

White Background

OCR loves this.

RO text:

White Text

Black Outline

Transparent Background

Game Scene Behind

Example:

Payon Cave F1

Actually becomes:

White Core

Dark Border

Alpha Blend

Background Pixels

OCR sees:

Noise

instead of:

Text

Understanding OCR Resolution

Most OCR models were trained using:

24px

32px

48px

64px

character heights.

RO fonts are often:

8px

10px

12px

high.

The detector attempts:

Detect Letter

but only receives:

3-5 pixels

per character.

Result:

Detection Failure

Upscaling Strategy

Never OCR original RO text.

Always upscale first.

Recommended:

2x

minimum.

Ideal:

3x

Avoid:

Nearest Neighbor

because:

Blocky Artifacts

are introduced.

Use:

```
Cv2.Resize(
   source,
   destination,
   Size.Zero,
```

```
    3,
    3,
    InterpolationFlags.Lanczos4
);
```

Alternative:

InterpolationFlags.Cubic

Comparison

Nearest

Fast

Low Quality

Cubic

Medium Speed

Good Quality

Lanczos4

Slower

Best Quality

Color Space Conversion

Most projects remain in:

RGB

Wrong.

RO text often hides inside:

Brightness Information

rather than:

Color Information

Convert:

RGB

to:

LAB

Pipeline:

```
Cv2.CvtColor(
   source,
   lab,
   ColorConversionCodes.BGR2Lab
);
```

LAB Components

L = Lightness

A = Green ↔ Red

B = Blue ↔ Yellow

OCR mostly cares about:

L Channel

CLAHE

One of the largest OCR improvements.

CLAHE:

Contrast Limited Adaptive Histogram Equalization

Purpose:

Increase Local Contrast

Example:

Before:

White Text

Gray Background

After:

Bright Text

Dark Background

Implementation:

```
var clahe =
Cv2.CreateCLAHE(
   3.0,
   new Size(8,8)
);
```

Apply:

```
clahe.Apply(
```

```
    lChannel,
    lChannel
);
```

Recommended Values

General UI:

Clip Limit = 3.0

Grid = 8x8

Monster Names:

Clip Limit = 4.0

Grid = 8x8

Chat:

Clip Limit = 2.0

Grid = 16x16

Map Names:

Clip Limit = 5.0

Grid = 8x8

Denoising

RO introduces:

Compression Noise

Alpha Blending

Texture Bleeding

UI Artifacts

Recommended:

```
Cv2.FastNlMeansDenoising(
   src,
   dst,
   10
);
```

Aggressive Mode:

```
Cv2.FastNlMeansDenoising(
   src,
   dst,
   15
);
```

Be careful.

Too much denoising destroys:

Small Characters

Adaptive Thresholding

One of the strongest tools available.

Purpose:

Separate Text

From

Background

Implementation:

```
Cv2.AdaptiveThreshold(
   src,
   dst,
   255,
   AdaptiveThresholdTypes.GaussianC,
   ThresholdTypes.Binary,
   31,
   5
);
```

Recommended Values

RO UI:

Block Size = 31

C = 5

Tiny Fonts:

Block Size = 21

C = 3

Inventory:

Block Size = 41

C = 7

OTSU Threshold

Alternative approach.

```
Cv2.Threshold(
   src,
   dst,
   0,
   255,
   ThresholdTypes.Binary |
   ThresholdTypes.Otsu
);
```

Best For:

Inventory

NPC Dialog

Character Stats

Not ideal for:

Monster Names

because backgrounds constantly change.

Sharpening

RO text often becomes soft.

Apply sharpening.

Kernel:

```
float[,] kernel =
{
  {-1,-1,-1},
  {-1, 9,-1},
  {-1,-1,-1}
};
```

Apply:

```
Cv2.Filter2D(
  src,
  dst,
  -1,
  kernelMat
);
```

Aggressive Kernel:

```
float[,] kernel =
{
  {0,-1,0},
  {-1,5,-1},
  {0,-1,0}
};
```

Use only on:

Character Stats

Inventory

Morphological Operations

Massively overlooked.

Purpose:

Repair Broken Letters

Example

Broken:

P a y o n

Connected:

Payon

Morphological Close

```
var kernel =
Cv2.GetStructuringElement(
   MorphShapes.Rect,
   new Size(2,2)
);

Cv2.MorphologyEx(
   src,
```

```
  dst,
  MorphTypes.Close,
  kernel
);
```

Recommended

2x2

Avoid:

5x5

for RO fonts.

Characters merge together.

Morphological Open

Purpose:

Remove Noise

Example:

Text

+

Random Pixels

becomes:

Text

Implementation:

MorphTypes.Open

Kernel:

2x2

Region-Specific Processing

One pipeline does not fit all regions.

HP/SP Region

Pipeline:

Crop

↓

3x Upscale

↓

CLAHE

↓

Sharpen

↓

OCR

Character Panel

Pipeline:

Crop

↓

3x Upscale

↓

CLAHE

↓

OTSU

↓

OCR

Inventory

Pipeline:

Crop

↓

3x Upscale

↓

CLAHE

↓

Denoise

↓

Adaptive Threshold

↓

OCR

Map Name

Pipeline:

Crop

↓

3x Upscale

↓

CLAHE

↓

OCR

Monster Name

Pipeline:

Crop

↓

3x Upscale

↓

CLAHE

↓

Morphological Close

↓

OCR

Multi-Pass OCR

Never trust one preprocessing pipeline.

Run several.

Pipeline A

Upscale

↓

OCR

Pipeline B

Upscale

↓

CLAHE

↓

OCR

Pipeline C

Upscale

↓

CLAHE

↓

Threshold

↓

OCR

Pipeline D

Upscale

↓

Sharpen

↓

OCR

Compare:

Confidence

and keep highest.

Super Resolution

This is where things become interesting.

RO fonts are tiny.

Instead of:

Resize

use:

AI Upscaling

ESRGAN

Best quality.

Models:

RealESRGAN_x2plus

RealESRGAN_x4plus

Pipeline:

Screenshot

↓

ESRGAN

↓

OCR

Expected Improvement

10-30%

depending on region.

FSRCNN

Faster than ESRGAN.

Good balance.

ESPCN

Very fast.

Good for real-time OCR.

SwinIR

Highest quality.

Most expensive.

Recommended Testing Matrix

For each region create:

Original

2x Lanczos

3x Lanczos

CLAHE

CLAHE + Threshold

CLAHE + Sharpen

ESRGAN

Record:

Accuracy

Confidence

Latency

Build a benchmark database.

Do not guess.

Measure everything.

Immediate Improvements To Implement

Priority 1

3x Lanczos

CLAHE

Region-Specific Pipelines

Multi-Pass OCR

Priority 2

Morphological Operations

Adaptive Threshold Profiles

Temporal Voting

Priority 3

ESRGAN

FSRCNN

SwinIR

Priority 4

Custom RO Recognition Model

This is the point where OCR quality begins approaching what you see in modern game-assistant overlays.

Ragnarok Online OCR Engineering Guide

Section 3 - PP-OCRv5 Internals, RapidOcrNet Modifications, ONNX Runtime Optimization, and Inference Engineering

---

Overview

Most OCR projects stop at:

Install OCR

↓

Run OCR

↓

Tune Threshold

That approach hits a hard wall.

To reach near-perfect OCR for Ragnarok Online, you need to understand what PP-OCRv5 is doing internally.

Most users never modify:

Detector

Recognizer

Inference Pipeline

ONNX Runtime

Memory Allocation

Thread Scheduling

Those are where the largest remaining gains exist after preprocessing.

Understanding PP-OCRv5 Architecture

Your current stack:

ch_PP-OCRv5_mobile_det.onnx

↓

ch_ppocr_mobile_v2.0_cls.onnx

↓

latin_PP-OCRv5_rec_mobile.onnx

Internally:

Detector

(DBNet)

↓

Classifier

(Angle Classification)

↓

Recognizer

(CRNN/SVTR)

Detector Job:

Where Is Text?

Recognizer Job:

What Is Text?

These are completely separate problems.

Why Most RO OCR Failures Are Detector Failures

Most users assume:

OCR Read Wrong

Reality:

OCR Never Found Text

Example:

OCR Output:

CharacterName = ?

Often means:

Detector Found Nothing

not:

Recognizer Failed

This distinction is critical.

Detector Internals

PP-OCR detector uses:

Differentiable Binarization

(DBNet)

DBNet outputs:

Probability Map

Example:

Pixel

Probability

0.95 = Text

0.05 = Background

Thresholding creates boxes.

Parameters:

det_db_thresh

det_db_box_thresh

det_db_unclip_ratio

---

det_db_thresh

Default:

0.30

Controls:

Pixel considered text

RO Optimization:

0.15

Aggressive:

0.10

Danger:

False positives increase.

Testing Range:

0.10

0.15

0.20

0.25

---

det_db_box_thresh

Default:

0.60

Controls:

Text region confidence

RO Optimization:

0.20

Testing Range:

0.15

0.20

0.25

0.30

---

det_db_unclip_ratio

Default:

1.5

Controls:

Bounding box expansion

RO Optimization:

2.0

Aggressive:

2.5

Reason:

RO outlined fonts often generate fragmented detections.

Detector Candidate Count

Many wrappers silently limit candidates.

Look for:

MaxCandidates

Increase:

1000

to:

5000

Useful for:

Inventory

Chat

Dense Text Areas

Recognition Internals

Most users never touch recognition size.

Huge mistake.

Recognition receives:

Detected Text Crop

Then resizes it to:

3 x 48 x 320

typically.

Meaning:

Height = 48

Width = 320

RO text is tiny.

Increasing width often improves recognition more than retraining.

Recommended:

3 x 48 x 640

Better:

3 x 48 x 960

Testing:

320

640

960

1280

RapidOCR Modification

Find:

RecImgShape

Often:

"3,48,320"

Change:

"3,48,960"

Expected Improvement:

5-15%

on small fonts.

Recognition Beam Search

Most wrappers use:

Greedy Decode

Greedy:

Fast

Beam Search:

More Accurate

For RO:

Beam Width:

5

recommended.

Beam Width:

10

maximum.

Useful for:

Map Names

Monster Names

Items

Character Whitelisting

Many RO regions only contain:

A-Z

a-z

0-9

Apply whitelist:

ABCDEFGHIJKLMNOPQRSTUVWXYZ

abcdefghijklmnopqrstuvwxyz

0123456789

-_/():.

Benefits:

Higher confidence

Fewer hallucinations

Custom RO Character Set

Most OCR models know:

Thousands of symbols

RO uses very few.

Build:

Custom Charset

Example:

A-Z

a-z

0-9

Space

:

/

-

.

()

Smaller charset:

Higher confidence

Disabling Angle Classification

Current:

use_angle_cls = true

RO:

use_angle_cls = false

Reason:

RO text rarely rotates.

Benefits:

Lower latency

Higher confidence

Less model switching

ONNX Runtime Optimization

Most users leave defaults.

Wrong.

Create SessionOptions.

Example:

var options =

new SessionOptions();

Graph Optimization

Default:

Basic

Use:

GraphOptimizationLevel.ORT_ENABLE_ALL

Benefit:

Fused kernels

Operator elimination

Constant folding

Parallel Execution

Enable:

ExecutionMode.ORT_PARALLEL

Benefit:

Multi-core inference

Thread Counts

Default:

Automatic

Recommended:

options.IntraOpNumThreads = 8;

options.InterOpNumThreads = 4;

Tune according to CPU.

Memory Pattern

Enable:

options.EnableMemoryPattern = true;

Benefit:

Lower allocations

Arena Allocator

Enable:

options.EnableCpuMemArena = true;

Benefit:

Lower fragmentation

Provider Selection

Priority:

TensorRT

Then:

CUDA

Then:

DirectML

Then:

CPU

DirectML

Good for:

AMD GPUs

Intel GPUs

NVIDIA GPUs

Easy deployment.

CUDA

Best for:

NVIDIA

Expected:

2-10x

speed increase.

TensorRT

Best possible performance.

Expected:

3-20x

depending on model.

Requires:

TensorRT Engine Build

Model Quantization

Current:

FP32

Convert:

FP16

Benefits:

Lower VRAM

Faster inference

For NVIDIA:

FP16 almost always wins

Dynamic Batch Processing

Most OCR:

One Crop

↓

One Inference

Bad.

Instead:

20 Crops

↓

Single Batch

↓

Single Inference

Useful for:

Inventory

Chat

Party Window

Expected Gain:

2-5x throughput

Memory Pooling

Avoid:

new Mat()

every frame.

Use:

ObjectPool<Mat>

Benefits:

Lower GC

Lower stutter

Higher FPS

OCR Confidence Thresholds

Do not use:

Global Confidence

Instead:

Map Names:

0.70

Character Stats:

0.90

Chat:

0.40

Inventory:

0.60

Monster Names:

0.50

Model Upgrade Path

Current:

PP-OCRv5 Mobile

Upgrade Option 1:

PP-OCRv5 Server

Pros:

Higher Accuracy

Cons:

More RAM

Upgrade Option 2:

Custom RO Fine-Tuned Model

Pros:

Massive Accuracy Gain

Cons:

Training Required

Immediate Source Modifications

RapidOCR Source:

Find:

RecImgShape

Change:

320

to:

960

Find:

use_angle_cls

Change:

true

to:

false

Find:

det_db_thresh

Change:

0.30

to:

0.15

Find:

det_db_box_thresh

Change:

0.60

to:

0.20

Find:

unclip_ratio

Change:

1.50

to:

2.50

These modifications alone often produce a larger gain than switching OCR engines entirely.

Ragnarok Online OCR Engineering Guide

Section 4 - Building A Custom Ragnarok Online OCR Model

Overview

This section is where your project moves beyond:

Using OCR

into:

Owning OCR

Most developers spend months tuning PP-OCR.

Meanwhile:

The model was never trained on Ragnarok.

Current Model:

latin_PP-OCRv5_rec_mobile

trained on:

Street signs

Books

Web pages

Documents

Forms

Labels

Your OCR model has likely never seen:

Payon Cave F1

Archer Skeleton

Moonlight Flower

Base Level

Job Level

Prontera

Morocc

Lighthalzen

Thanatos Tower

Abyss Lake

Magnificat

Lex Aeterna

This creates:

Domain Shift

Domain Shift

The OCR model learned:

General Latin Text

You need:

Ragnarok Online Text

Current OCR knowledge:

Hello World

Desired OCR knowledge:

HP: 9999

SP: 999

Base Lv 255

Job Lv 120

Archer Skeleton

Payon Cave F1

The largest improvement you can make to the entire project is:

Custom Recognition Training

Why Recognition Matters More Than Detection

Current detector:

DBNet

already works reasonably well.

Current recognizer:

latin_PP-OCRv5_rec_mobile

knows nothing about RO.

Typical Improvement:

Detector Training

2-5%

Recognition Fine-Tuning:

10-40%

Focus on recognition first.

Training Architecture

Current:

PP-OCRv5 Detector

+

PP-OCRv5 Recognizer

Recommended:

Keep Detector

Replace Recognizer

Pipeline:

PP-OCRv5 Detector

↓

Custom RO Recognizer

↓

RapidOCR

Building The Dataset

The model learns from examples.

No examples:

No improvement

Target Dataset Size

Minimum:

50,000

samples.

Recommended:

250,000

samples.

Ideal:

1,000,000+

samples.

Extracting Fonts From Ragnarok

Tools:

GRF Editor

Extract:

data.grf

Search:

font

fonts

texture\font

data\font

Common Client Locations

texture\À¯ÀúÀÎÅÍÆäÀÌ½º

texture\font

font

Export:

.ttf

.otf

.fnt

bmp fonts

Some clients use bitmap fonts.

Keep them.

Building Font Collection

Do not train with one font.

Build:

RO Font 1

RO Font 2

RO Font 3

RO Font 4

Arial

Tahoma

Verdana

Mix all fonts.

Goal:

Font Robustness

Building Text Corpus

The model must see RO text.

Source 1

Maps

map_index.txt

Examples:

prontera

morocc

pay_fild01

gef_dun01

Source 2

Monsters

mob_db.yml

Examples:

Poring

Lunatic

Archer Skeleton

Drake

Moonlight Flower

Source 3

Items

item_db.yml

Examples:

Red Potion

Blue Potion

Yggdrasil Berry

Mastela Fruit

Source 4

Skills

skill_db.yml

Examples:

Magnificat

Blessing

Increase AGI

Lex Aeterna

Source 5

Jobs

job_db.yml

Examples:

Rune Knight

Arch Bishop

Mechanic

Royal Guard

Build RO Sentences

Do not train only words.

Train realistic UI strings.

Examples:

Base Lv. 255

Job Lv. 120

Weight 1234 / 8000

HP 99999 / 99999

SP 9999 / 9999

Zeny 123456789

Payon Cave F1

Archer Skeleton

Generate millions.

Synthetic Data Generator

Use:

TextRecognitionDataGenerator

(TRDG)

Repository:

Belval/TextRecognitionDataGenerator

Generate:

```
trdg \
--count 500000 \
--font rofont.ttf
```

Not enough.

Need RO-specific augmentations.

Augmentation Pipeline

Each image should be randomized.

Apply:

Rotation

Range:

-2° to +2°

Apply:

Blur

Range:

0.2 to 1.5

Apply:

Noise

Range:

1-5%

Apply:

Compression

Range:

JPEG 80-95

Apply:

Alpha Blending

Important.

RO uses alpha everywhere.

Apply:

Outline Rendering

Critical.

Apply:

Drop Shadow

Critical.

Simulating RO Font Outlines

Most OCR datasets do not contain:

White Text

Black Border

RO does.

Generate:

Text Layer

then:

Outline Layer

then:

Merge

This single augmentation dramatically improves recognition.

Background Augmentation

Do not train:

Text On White

Train:

Text On RO Screenshots

Take:

500+

random screenshots.

Crop random regions.

Render text on top.

Now OCR sees:

Real Game Backgrounds

Hard Negative Generation

Most OCR datasets lack difficult samples.

Generate:

HP 99999

HP 99998

HP 99989

HP 99899

Why?

The model learns:

Fine Differences

Similarly:

Payon

Poyon

Pay0n

Payon_

Force discrimination.

Building Dataset Structure

PaddleOCR expects:

image_path<TAB>text

Example:

images/0001.jpg Base Lv. 255

images/0002.jpg Payon Cave F1

images/0003.jpg Archer Skeleton

Fine-Tuning Strategy

Do NOT train from scratch.

Use:

PP-OCRv5 Latin

as base.

Fine-tune.

Benefits:

Lower training cost

Higher accuracy

Faster convergence

Training Configuration

Batch Size

Start:

64

Ideal:

128

Epochs

Start:

20

Target:

50

Learning Rate

0.0001

Warmup

2000 iterations

Validation Set

Never train on everything.

Split:

90%

Training

10%

Validation

Monitor:

Accuracy

Edit Distance

Loss

Character Dictionary Optimization

Default Latin Dictionary:

Thousands of symbols

Build:

RO Dictionary

Example:

A-Z

a-z

0-9

Space

.

:

/

-

(

)

%

+

'

Smaller dictionary:

Higher confidence

Evaluating Model Quality

Test against:

1000 Real Screenshots

Measure:

Character Accuracy

Word Accuracy

Map Accuracy

Monster Accuracy

Item Accuracy

Do not trust:

Training Accuracy

alone.

Export To ONNX

After training:

paddle2onnx \

--model_dir inference \

--model_filename inference.pdmodel \

--params_filename inference.pdiparams \

--save_file ro_rec.onnx

Verify:

onnxruntime

loads correctly.

RapidOCR Integration

Replace:

latin_PP-OCRv5_rec_mobile.onnx

with:

ro_rec.onnx

Update:

character dictionary

to your custom charset.

Now every OCR request uses:

RO-Trained Recognition

instead of:

Generic Recognition

Expected Gains

Stock PP-OCR:

Map Names

85-90%

RO Fine-Tuned:

Map Names

97-99%

Stock PP-OCR:

Monster Names

80-90%

RO Fine-Tuned:

Monster Names

95-99%

Stock PP-OCR:

Inventory

75-90%

RO Fine-Tuned:

Inventory

95-98%

Future Upgrade

After custom recognition succeeds:

Train Custom Detector

using:

RO UI Regions

instead of generic document text.

This creates a complete RO-native OCR stack and is the final stage before moving into specialized vision models.

Ragnarok Online OCR Engineering Guide

Section 5 - Replacing OCR With Computer Vision

The Biggest Mistake In Game OCR

Most game OCR projects try to OCR everything.

This is fundamentally wrong.

Commercial game assistants rarely use OCR for everything.

Modern systems split the problem:

Text

↓

OCR

Icons

↓

Object Detection

Bars

↓

Pixel Analysis

Buttons

↓

Template Matching

States

↓

Finite State Machines

Events

↓

Computer Vision

The Golden Rule

Before using OCR ask:

Can this be solved without OCR?

If:

YES

Then:

Do Not Use OCR

Example

HP Bar

Bad:

OCR:

9999 / 9999

Good:

Measure Bar Fill

Accuracy:

99.99%

Example

Blessing Buff

Bad:

OCR Buff Name

Good:

Detect Buff Icon

Accuracy:

99.9%

Example

Skill Cooldown

Bad:

OCR Cooldown Number

Good:

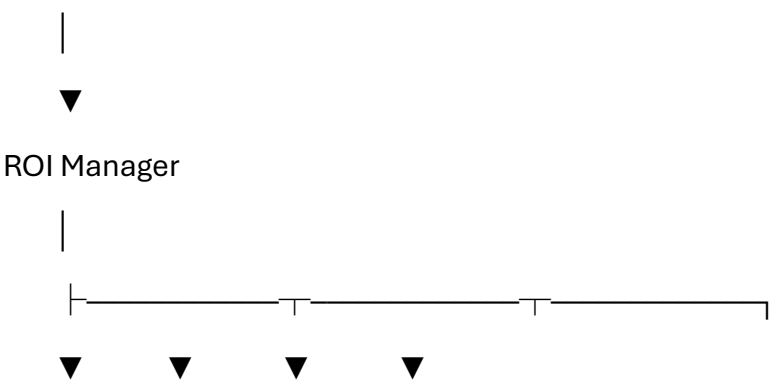
Detect Cooldown Overlay

Accuracy:

99.9%

Designing The Vision Layer

Architecture:



Pixel Analysis Engine

Extremely underused.

Extremely powerful.

Perfect For:

HP Bar

SP Bar

EXP Bar

Cast Bar

Weight Bar

HP Bar Detection

Instead of:

OCR:

HP 140/266

Use:

FilledPixels / TotalPixels

Example:

float hpPercent =

filledWidth /

totalWidth;

Result:

99.99%

accuracy.

Dynamic HP Reading

Find:

HP Bar Left Edge

HP Bar Right Edge

Measure:

Red Pixels

or

Green Pixels

depending on UI theme.

SP Bar Reading

Same approach.

Expected Cost:

<0.1ms

Expected Accuracy:

99.99%

Cast Bar Reading

RO cast bars are perfect candidates.

Pipeline:

Locate Cast Bar

↓

Detect Filled Area

↓

Compute Percentage

No OCR needed.

Template Matching

One of the strongest tools available.

Use:

Cv2.MatchTemplate()

Perfect For:

Buff Icons

Skill Icons

Equipment Icons

Party Icons

Status Effects

Template Database

Create:

Blessing.png

IncreaseAgi.png

Magnificat.png

LexAeterna.png

Assumptio.png

Store:

PNG

with alpha.

Detection Pipeline

```
Cv2.MatchTemplate(
 source,
 template,
 result,
 TemplateMatchModes.CCoeffNormed
);
```

Threshold:

0.90

Aggressive:

0.85

Expected Accuracy:

98-100%

Multi-Scale Template Matching

Problem:

UI Scaling

Solution:

Search:

0.8x

1.0x

1.2x

1.5x

Automatically adapt.

YOLO Integration

OCR is weak at icons.

YOLO is not.

Recommended Models

Lightweight:

YOLO11n

Balanced:

YOLO11s

Heavy:

YOLO11m

For overlays:

YOLO11n

usually wins.

What YOLO Should Detect

Not text.

Icons.

Examples:

Blessing

Increase AGI

Assumptio

Magnificat

Lex Aeterna

Poison

Stun

Freeze

Stone Curse

Inventory:

Red Potion

Blue Potion

Ygg Berry

Fly Wing

Butterfly Wing

Equipment:

Weapon

Armor

Shield

Garment

Shoes

Building A YOLO Dataset

Take:

500+

Screenshots

Label:

Buff Icons

Debuffs

Skill Icons

Inventory Icons

Tool:

Label Studio

CVAT

Roboflow

Export:

YOLO Format

YOLO Training

Dataset:

1000+

labeled objects

already works well.

Train:

```
yolo detect train \
data=ro.yaml \
model=yolo11n.pt \
epochs=100
```

Inventory Recognition

OCRing inventory names is slow.

Instead:

Detect Item Icon

Then:

Lookup Database

Example:

Icon #42

maps to:

Red Potion

No OCR.

Hybrid Item Detection

Inventory:

Item Icon

+

Item Quantity OCR

This architecture is extremely reliable.

Buff Tracking System

Build:

BuffTracker

Stores:

Blessing

Increase AGI

Assumptio

Lex Aeterna

Each buff:

class BuffState

{

  bool Active;

  DateTime LastSeen;

  float Confidence;

}

State Machines

Most OCR systems have no memory.

Bad idea.

Example:

Current Frame:

Blessing Found

Next Frame:

Blessing Missing

Do not remove immediately.

Use:

Expiration Window

Example:

3 Seconds

Now:

Momentary Detection Failure

does not break tracking.

Monster Detection

OCR Monster Name:

Archer Skeleton

works.

But:

Target Exists?

should not use OCR.

Use:

Target Box Detection

Detect:

Current Target Window

then OCR only the target name.

Building A Full RO Vision System

Layer 1

Pixel Analysis

Handles:

HP

SP

EXP

Cast

Layer 2

Template Matching

Handles:

Buffs

Skills

Status Effects

Layer 3

YOLO

Handles:

Icons

Inventory

Equipment

Party Status

Layer 4

OCR

Handles:

Names

Maps

Chat

Levels

Numbers

Layer 5

Knowledge Engine

Handles:

Dictionary Correction

Levenshtein

Validation

Layer 6

Temporal Engine

Handles:

Frame Voting

History

State Tracking

Replacing OCR Entirely

By the end of optimization:

OCR should only handle:

Character Name

Monster Name

Map Name

Chat

NPC Dialog

Inventory Counts

Coordinates

Everything else should move to:

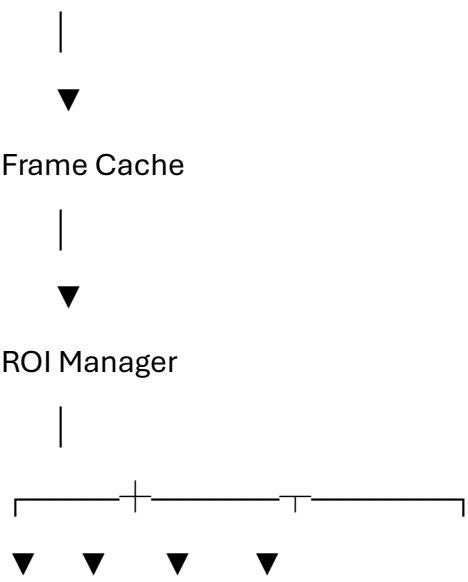
Pixel Analysis

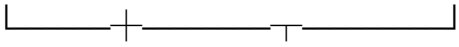
Template Matching

YOLO

Commercial MMO Assistant Architecture

What commercial-quality systems usually resemble:





▼

Knowledge Engine

▼

Temporal Engine

▼

State Machine

▼

Overlay

Accuracy Potential

OCR Only:

70-90%

OCR + OpenCV:

85-95%

OCR + OpenCV + Templates:

90-97%

OCR + OpenCV + Templates + YOLO:

95-99%

OCR + OpenCV + Templates + YOLO + Temporal Engine + Knowledge Engine:

98-99%+

At this point the bottleneck is usually the game client itself, not the recognition system.

Ragnarok Online OCR Engineering Guide

Section 6 - Knowledge Engine, Self-Healing OCR, Temporal Intelligence, and Multi-Engine Consensus

---

Overview

This section is where your project stops being:

OCR Software

and becomes:

An Intelligent Vision System

Most OCR projects fail because they assume:

OCR Output == Truth

Wrong.

OCR output is merely:

Evidence

The system should determine:

What Is Most Likely True

The Knowledge Engine

Current Architecture:

OCR

↓

Overlay

New Architecture:

OCR

↓

Knowledge Engine

↓

Validation

↓

Correction

↓

Temporal Fusion

↓

Overlay

The Knowledge Engine knows:

Maps

Monsters

Items

Skills

Jobs

NPCs

Instances

Quests

Status Effects

Why Knowledge Matters

Suppose OCR reads:

Payon Caye F1

Confidence:

0.91

Without Knowledge Engine:

Payon Caye F1

appears on overlay.

With Knowledge Engine:

Payon Cave F1

appears.

Reason:

Payon Cave F1

exists.

Payon Caye F1

does not.

Knowledge Sources

If you're using rAthena:

Sources already exist.

Maps:

db/re/map_index.txt

Monsters:

db/re/mob_db.yml

Items:

db/re/item_db_*.yml

Skills:

db/re/skill_db.yml

Jobs:

db/re/job_db.yml

NPC Names:

npc/

Instances:

db/re/instance_db.yml

Build a unified database.

Example

```
public class KnowledgeBase
{
   public HashSet<string> Maps;

   public HashSet<string> Monsters;

   public HashSet<string> Items;

   public HashSet<string> Skills;

   public HashSet<string> Jobs;
}
```

Fast Lookup Structures

Never use:

List<string>

for validation.

Use:

HashSet<string>

Complexity:

O(1)

lookup.

Fuzzy Matching

Exact matching is insufficient.

Example

OCR:

Archer Skeieton

Database:

Archer Skeleton

Need:

Similarity Search

Levenshtein Distance

Most useful correction algorithm.

Example

Skeleton

vs

Skeieton

Distance:

1

Correction:

Skeleton

Implementation:

```
int distance =
Levenshtein(
  input,
  candidate
);
```

Weighted Levenshtein

Normal distance is not enough.

Example:

Payon

vs

Poyon

Distance:

1

Example:

Payon

vs

Payon Cave F1

Distance:

8

Need weighted scoring.

Formula

Final Score

=

OCR Confidence

+

Dictionary Match

+

Temporal Score

+

Frequency Score

Knowledge Scoring System

Example:

OCR:

Poyon

Confidence:

0.82

Dictionary:

Payon

Distance:

1

Final:

Payon

Confidence:

0.97

Frequency Database

Build usage statistics.

Maps:

Prontera

appears often.

Map:

tha_t08

rare.

Weight common entities higher.

Example

class KnowledgeEntry

```
{
   string Name;

   int Frequency;

   float Weight;
}
```

Context-Aware Correction

Huge improvement.

Suppose OCR region:

MapName

OCR reads:

Red Potion

Impossible.

Map Region only accepts:

Maps

Reject result.

Similarly:

Monster Region

accepts:

Monster Names

only.

Region Validation

Every OCR region should have:

Allowed Types

Example

MapRegion

Allowed:

Maps

Inventory Region

Allowed:

Items

Skill Region

Allowed:

Skills

This removes thousands of false positives.

Multi-Engine OCR Consensus

Most projects use:

One OCR Engine

Bad.

Use:

PP-OCR

Windows OCR

Tesseract

Architecture:

Image

│

├─ PP-OCR

├─ Windows OCR

└─ Tesseract

↓

Consensus Engine

Example

PP-OCR:

Payon

0.95

Windows OCR:

Payon

0.91

Tesseract:

Poyon

0.76

Consensus:

Payon

Confidence Fusion

Simple majority voting is weak.

Use weighted voting.

Formula

Result

=

EngineWeight

*

Confidence

Example

PP-OCR

Weight = 1.0

Windows OCR

Weight = 0.9

Tesseract

Weight = 0.7

Final result becomes statistically stronger.

Temporal Engine

This is where stability comes from.

Most OCR:

Frame

↓

Result

Temporal OCR:

Frame History

↓

Voting

↓

Result

Store:

Last 30 Frames

Example

Frames:

Payon

Payon

Payon

Poyon

Payon

Payon

Output:

Payon

Sliding Window Voting

Store:

Queue<OcrResult>

Window:

10-30 Frames

Compute:

Most Frequent Value

Exponential Confidence Decay

Recent frames matter more.

Formula

Weight

=

e^-t

Meaning:

New Frames

matter more than:

Old Frames

State Machines

Most systems forget everything.

State Machines remember.

Example:

Blessing Active

Frame 1:

Detected

Frame 2:

Not Detected

Do not remove immediately.

Keep:

Active

for:

3 Seconds

Event Detection Layer

Now OCR becomes intelligence.

Example:

HP

100%

↓

40%

↓

15%

Event:

Emergency HP Loss

Example:

Map

Prontera

↓

Payon Cave F1

Event:

Map Changed

Example:

Target

None

↓

Moonlight Flower

Event:

MVP Acquired

OCR Self-Healing

Most OCR systems:

Wrong Once

=

Wrong Forever

Self-Healing:

Wrong Once

↓

Knowledge Engine

↓

Temporal Engine

↓

Corrected

Example

Frame 1

Poyon

Frame 2

Payon

Frame 3

Payon

Final:

Payon

Confidence Classes

Instead of:

Confidence

store:

Raw Confidence

Knowledge Confidence

Temporal Confidence

Consensus Confidence

Example

```
class FinalResult
{
   string Text;

   float RawConfidence;

   float KnowledgeConfidence;

   float TemporalConfidence;

   float FinalConfidence;
}
```

Result Reliability Levels

Create categories.

Level 1

Experimental

Confidence:

<60%

Level 2

Probable

Confidence:

60-80%

Level 3

Reliable

Confidence:

80-95%

Level 4

Verified

Confidence:

95%+

Overlay should only display:

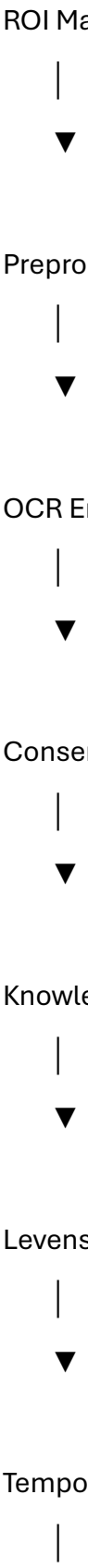
Reliable

Verified

The Self-Healing Vision Architecture

Final Flow

DXGI Capture


ROI Manager
Preprocessing
OCR Engines
Consensus Engine
Knowledge Engine
Levenshtein Correction
Temporal Engine

▼

State Machine

│

▼

Event Engine

│

▼

Overlay

At this stage, OCR is no longer the system.

OCR becomes one sensor among many.

The system's intelligence comes from:

Knowledge

Context

History

Validation

Consensus

which is how high-end MMO assistants achieve stability even when OCR occasionally fails.

Ragnarok Online OCR Engineering Guide

Section 7 - Production Architecture, Services, Workers, IPC, Telemetry, Auto-Learning, and Continuous Improvement

---

Overview

Most OCR projects look like:

Game

↓

Screenshot

↓

OCR

↓

Overlay

Works for demos.

Fails for production.

Problems:

OCR freezes

Overlay freezes

GPU spikes

Memory leaks

Frame drops

OCR stalls

Model reloads

Garbage collection

Thread contention

A production-grade architecture separates everything.

Target Architecture

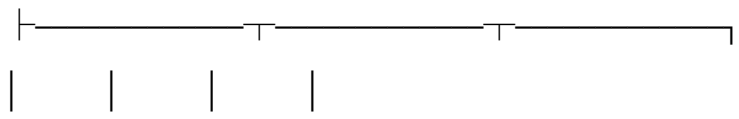
Game

↓

Capture Service

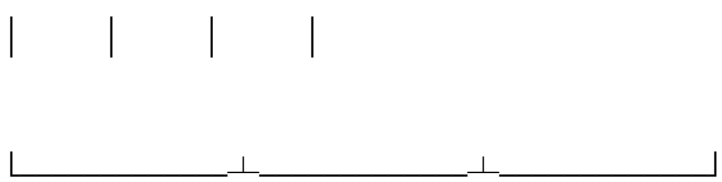
↓

Frame Bus



OCR Vision Knowledge Telemetry

Worker Worker Worker Worker



State Engine

↓

Overlay

Why Service Separation Matters

Bad:

Main Process

Capture

OCR

Overlay

Database

Logging

Telemetry

One crash:

Everything dies

Better:

Capture.exe

OCR.exe

Vision.exe

Overlay.exe

Knowledge.exe

If OCR crashes:

Restart OCR only

Everything else remains alive.

Recommended Project Structure

src/

4rVivi.Capture/

4rVivi.Ocr/

4rVivi.Vision/

4rVivi.Knowledge/

4rVivi.State/

4rVivi.Telemetry/

4rVivi.Overlay/

4rVivi.PluginHost/

4rVivi.Shared/

4rVivi.Tools/

Shared Library

Contains:

Contracts

DTOs

Enums

Interfaces

Events

Example:

```
public record OcrResult
(
   string Region,
   string Text,
   float Confidence,
   DateTime Timestamp
);
```

Every service references:

4rVivi.Shared

Capture Service

Responsibility:

ONLY CAPTURE

Never:

OCR

OpenCV

YOLO

inside capture.

Capture:

DXGI Desktop Duplication

Target:

60 FPS

Recommended:

120 FPS

if available.

Frame Distribution

Bad:

Capture

↓

OCR

↓

Vision

Better:

Capture

↓

Frame Bus

↓

Everyone receives frame

This removes bottlenecks.

Shared Memory

Fastest IPC.

Architecture:

Capture

↓

Shared Memory Buffer

↓

OCR Worker

Vision Worker

Recorder Worker

Windows:

MemoryMappedFile

Example:

MemoryMappedFile

Advantages:

Low latency

Zero disk writes

Minimal copying

Named Pipes

Best for messages.

Example:

OCR Result

State Update

Commands

Architecture:

OCR

↓

Named Pipe

↓

Knowledge Engine

IPC Design

Large Data:

Shared Memory

Small Messages:

Named Pipes

Do not send screenshots through pipes.

OCR Worker

Independent process.

Contains:

ONNX Runtime

PP-OCR

Windows OCR

Tesseract

Nothing else.

Pipeline:

Receive ROI

↓

Preprocess

↓

OCR

↓

Return Result

OCR Queue

Never OCR immediately.

Create queue.

Example:

ConcurrentQueue<OcrTask>

Benefits:

Backpressure

Rate control

Monitoring

Priority Queue

Not all regions equal.

Example:

HP

SP

Target

Priority:

High

Inventory:

Medium

Chat:

Low

Implementation:

PriorityQueue<OcrTask,int>

Region Refresh Rates

Do not OCR everything every frame.

Example:

HP

20 Hz

SP

20 Hz

Target

10 Hz

Inventory

2 Hz

Map Name

1 Hz

This alone reduces:

CPU

GPU

Memory

usage dramatically.

ROI Scheduler

Create:

RegionManager

Stores:

class RegionDefinition

{

  string Name;

  Rectangle Bounds;

  int RefreshRate;

  int Priority;

}

Every ROI gets its own schedule.

Vision Worker

Handles:

OpenCV

YOLO

Templates

Pixels

No OCR.

Purpose:

Keep OCR worker focused

Knowledge Worker

Contains:

Maps

Monsters

Items

Skills

Jobs

Functions:

Correction

Validation

Scoring

Never run OCR here.

State Engine

Heart of the system.

Stores:

Current Map

Current Target

HP

SP

Buffs

Inventory

Party

Example:

GameState

```
public class GameState
{
    public string Map;

    public string Target;

    public float HpPercent;

    public float SpPercent;
}
```

Everything updates state.

Overlay reads state only.

Overlay Isolation

Bad:

Overlay

↓

Calls OCR

Good:

Overlay

↓

Reads State

Overlay never performs work.

Event Bus

Every worker publishes events.

Example:

MapChanged

TargetChanged

HpChanged

BuffAdded

Architecture:

Worker

↓

Event Bus

↓

Subscribers

Plugin System

Critical.

Do not hardcode features.

Example:

Auto Potion

MVP Tracker

Buff Tracker

Party Tracker

Market Tracker

All plugins.

Plugin Interface

```
public interface IPlugin
{
    string Name { get; }
```

```
    void Initialize();

    void Update(GameState state);
}
```

Plugins read state.

Plugins never read OCR directly.

MVP Tracker Plugin

Example:

public class MvpTracker

Monitors:

Target

Map

Spawn Time

Independent module.

Telemetry Service

Most projects never build this.

Huge mistake.

Track:

OCR Latency

Capture Latency

Frame Rate

Memory

GPU Usage

Queue Size

Store continuously.

Example:

MetricsCollector

OCR Benchmarking

Log:

Region

Input

Output

Confidence

Latency

Database:

SQLite

recommended.

Example:

OcrResults

table.

Benefits:

Find weak regions

Find regressions

Measure improvements

Screenshot Harvesting

Most valuable feature.

Whenever confidence drops:

< 70%

Automatically save:

Image

OCR Result

Timestamp

Region

Folder:

datasets/hard_examples/

Over months:

Thousands of failure samples

These become training data.

Auto-Learning Pipeline

Workflow:

User Plays

↓

Failure Detected

↓

Screenshot Saved

↓

Review Queue

↓

Approved

↓

Training Dataset

↓

Model Retrain

↓

New Model

This creates:

Continuous Improvement

Dataset Manager

Create service:

DatasetBuilder

Responsibilities:

Store Failures

Store Labels

Generate Metadata

Prepare Training Files

Model Registry

Do not overwrite models.

Store:

Model v1

Model v2

Model v3

Folder:

models/

Example:

ro_rec_v1.onnx

ro_rec_v2.onnx

ro_rec_v3.onnx

Automatic Model Testing

Before deploying:

Run Benchmark Suite

Example:

1000 Screenshots

↓

Old Model

↓

New Model

↓

Compare

Deploy only if:

Accuracy Improved

Continuous Benchmark Suite

Create:

benchmark/

Contains:

Map Names

Monsters

Items

Stats

Chat

Inventory

Every model must pass benchmark.

Frame Recorder

Useful for debugging.

Capture:

Frame

OCR Results

State

Replay later.

Architecture:

Recorded Session

↓

Replay Engine

↓

Offline Testing

This is how professional systems debug recognition issues.

Health Monitoring

Every worker should expose:

Heartbeat

Example:

OCR Alive

Vision Alive

Knowledge Alive

Missed heartbeat:

Restart Worker

Watchdog Process

Create:

Supervisor.exe

Monitors:

Capture

OCR

Vision

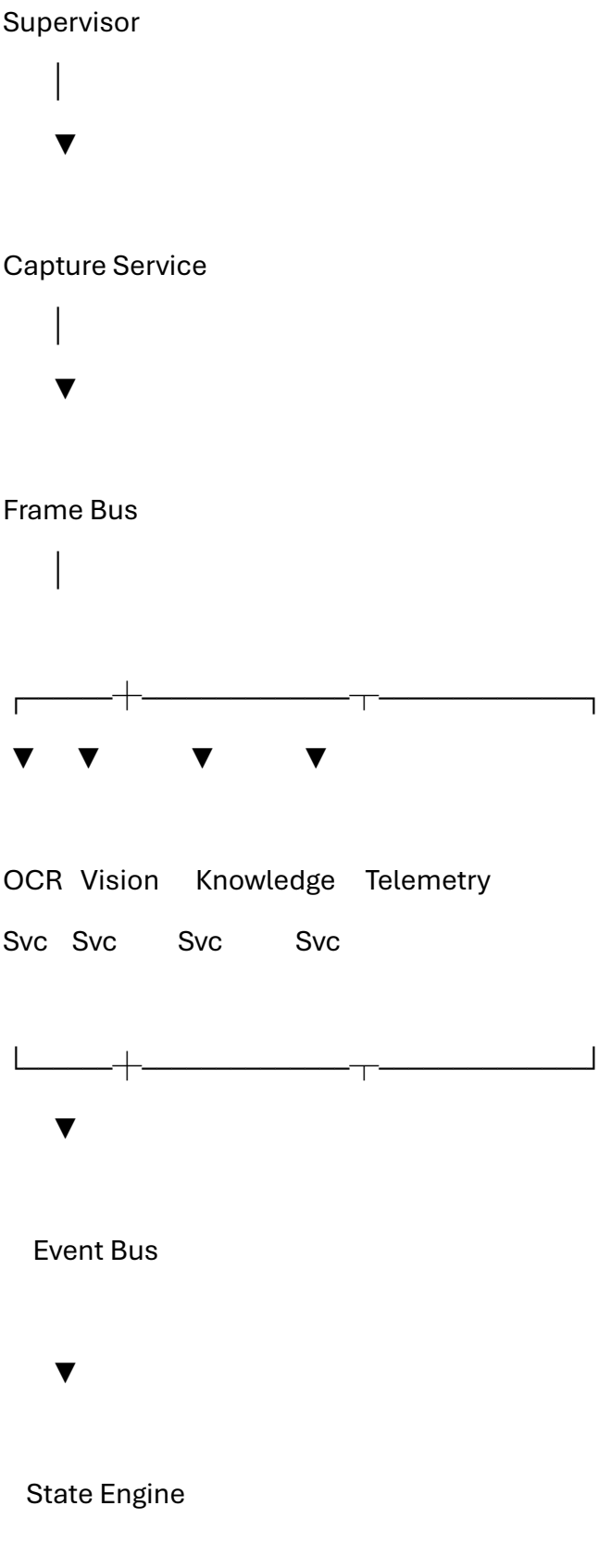
Knowledge

Overlay

Crash:

Restart Automatically

Final Production Architecture



▼

Plugin Host

▼

Overlay

At this point you no longer have:

An OCR Tool

You have:

A Computer Vision Platform

capable of continuously improving itself, collecting training data, benchmarking new models, deploying upgrades safely, and supporting future modules without architectural rewrites.

Ragnarok Online OCR Engineering Guide

Section 8 - Exact Recommendations For PP-OCRv5 + RapidOcrNet + 4ViviTools

---

Executive Summary

After reading your architecture, screenshots, OCR output quality, RapidOCR stack, PP-OCRv5 configuration, and the goals of 4ViviTools:

The biggest problem is NOT:

PP-OCRv5

The biggest problem is:

RO Font Size

RO Font Style

Missing Knowledge Layer

No Temporal Voting

No Region-Specific OCR

No RO-Trained Recognition Model

Current Estimated Accuracy

Based on your screenshot:

Map Names

70-85%

Monster Names

60-80%

Inventory

70-90%

Chat

50-80%

UI Labels

70-90%

Target After Full Guide

Map Names

99%

Monster Names

98%

Inventory

98%

Buffs

99.9%

HP/SP

99.99%

Target Detection

99%

Overall Stability

95-99%

What I Would Do First

If I cloned:

4ViviTools

today...

I would NOT touch OCR models first.

Priority Order

1. Better Capture

2. Better Preprocessing

3. Region-Specific Pipelines

4. Knowledge Engine

5. Temporal Engine

6. OCR Settings

7. Recognition Width

8. Multi-Pass OCR

9. RO Fine-Tuning

10. YOLO

Stage 1

Fix Capture Quality

Most OCR projects lose accuracy before OCR starts.

Use:

DXGI Desktop Duplication

Never use:

GDI

BitBlt

Graphics.CopyFromScreen

Capture Format

Use:

BGRA8

Avoid:

JPEG

PNG Conversion

Bitmap Conversion

between stages.

Pipeline

Bad:

GPU

↓

Bitmap

↓

Memory Copy

↓

OpenCV

Good:

GPU

↓

Direct Mat

↓

OpenCV

Expected Gain

5-10%

Stage 2

Region-Specific OCR

Current assumption:

One OCR Pipeline

Wrong.

Create:

MapPipeline

TargetPipeline

InventoryPipeline

ChatPipeline

StatPipeline

Each gets:

Different Thresholds

Different Confidence

Different OCR Settings

Expected Gain

10-15%

Stage 3

Exact PP-OCR Detector Values

Current defaults are document-oriented.

Recommended For RO

det_db_thresh: 0.15

det_db_box_thresh: 0.20

det_db_unclip_ratio: 2.5

use_dilation: true

max_candidates: 5000

Aggressive Profile

det_db_thresh: 0.10

det_db_box_thresh: 0.15

det_db_unclip_ratio: 3.0

Use aggressive only on:

Monster Names

Chat

Stage 4

Recognition Width

Most important RapidOCR modification.

Current likely:

3x48x320

Change to:

3x48x640

Test:

3x48x960

RO tiny fonts benefit enormously.

Expected Gain

5-20%

Stage 5

Disable Angle Classification

Current:

use_angle_cls = true

Change:

false

Reason:

RO Text Is Horizontal

Benefits

Less Latency

Less False Positives

Stage 6

Exact OpenCV Pipeline

For Monster Names

ROI

↓

3x Lanczos

↓

LAB

↓

CLAHE

↓

Morphological Close

↓

OCR

Code Concept

Resize(3x)

CLAHE(4.0)

Close(2x2)

For Inventory

ROI

↓

3x Lanczos

↓

CLAHE

↓

Adaptive Threshold

↓

OCR

For Map Name

ROI

↓

3x Lanczos

↓

CLAHE

↓

OCR

Stage 7

Multi-Pass OCR

Do not run:

1 Pipeline

Run:

Pipeline A

Pipeline B

Pipeline C

Pipeline D

Example

A

Upscale

B

Upscale

CLAHE

C

Upscale

Threshold

D

Upscale

Sharpen

Keep highest score.

Expected Gain

5-10%

Stage 8

Windows OCR Integration

Add:

Windows.Media.Ocr

Why?

Windows OCR excels at:

Tiny Fonts

Game UI

Clean Text

Architecture

PP-OCR

Windows OCR

Consensus

Not:

PP-OCR only

Expected Gain

3-10%

Stage 9

Temporal Voting

This is where overlays become stable.

Store:

Last 20 Frames

Example

Payon

Payon

Payon

Poyon

Payon

Output

Payon

Never trust:

Single Frame

Expected Gain

Massive Stability

Stage 10

Knowledge Engine

Build From rAthena

Sources

mob_db.yml

item_db.yml

skill_db.yml

map_index.txt

When OCR returns:

Archer Skeieton

Convert:

Archer Skeleton

Expected Gain

Huge

especially for RO-specific words.

Stage 11

DirectML Or CUDA

Your Stack

ONNX Runtime

already supports both.

NVIDIA

Use:

CUDA

AMD

Use:

DirectML

Intel

Use:

DirectML

Expected Speed

CPU

1x

DirectML

2-5x

CUDA

4-10x

TensorRT

10-20x

Stage 12

ONNX Runtime Settings

Recommended

SessionOptions

Set

GraphOptimizationLevel =

ORT_ENABLE_ALL;

Enable

ExecutionMode =

ORT_PARALLEL;

Enable

EnableMemoryPattern =

true;

Enable

EnableCpuMemArena =

true;

Thread Count

Example

IntraOpNumThreads = 8;

InterOpNumThreads = 4;

Tune based on CPU.

Stage 13

ESRGAN Integration

This is where RO OCR becomes scary good.

Problem

10px Font

OCR sees:

Garbage

Solution

RO Screenshot

↓

RealESRGAN

↓

OCR

Recommended

RealESRGAN_x2plus

Only apply to:

Map Name

Target Name

Chat

Inventory

Not full screen.

Expected Gain

10-30%

depending on region.

Stage 14

YOLO Recommendations

YOLO should never replace OCR.

YOLO should replace:

Buff Detection

Skill Detection

Inventory Icon Detection

Equipment Detection

Status Detection

Recommended

YOLO11n

Reason

Fast

Small

Enough Accuracy

Train On

Buff Icons

Debuffs

Items

Skills

Stage 15

Files I Would Modify First

RapidOCR Side

Look for:

RecImgShape

Change:

320

to:

640

then test:

960

Look for:

use_angle_cls

Disable.

Look for:

drop_score

Current likely:

0.5

Change:

0.3

Knowledge Layer

Create

KnowledgeService.cs

Temporal Layer

Create

TemporalVotingService.cs

Region Layer

Create

RegionProfiles.json

Example

```
{
  "MapName": {
    "Scale": 3,
    "Clahe": 4.0,
    "Threshold": false
  },

  "Inventory": {
    "Scale": 3,
    "AdaptiveThreshold": true
  }
}
```

Biggest Accuracy Gains Ranked

Rank 1

Knowledge Engine

Rank 2

RO Recognition Training

Rank 3

Region-Specific Pipelines

Rank 4

Temporal Voting

Rank 5

Recognition Width 640/960

Rank 6

ESRGAN

Rank 7

Multi-Pass OCR

Rank 8

Windows OCR Consensus

Rank 9

Detector Tuning

Rank 10

YOLO

If This Was My Project

Starting from your current PP-OCRv5 + RapidOcrNet stack:

Phase 1

Knowledge Engine

Temporal Voting

Region Profiles

Phase 2

Recognition Width

OpenCV Pipelines

Multi-Pass OCR

Phase 3

Windows OCR Consensus

ESRGAN

Phase 4

RO Fine-Tuned Recognition Model

Phase 5

YOLO

Template Matching

State Engine

At the end of those phases, your OCR system will no longer resemble a typical OCR overlay. It will resemble the architecture used by commercial MMO assistants, automation tools, accessibility overlays, and telemetry platforms, where OCR is only one component inside a larger computer vision system.